Handwritten Digit Classification System Powered by FPGA

Project Report

IC Project-1: Group 7

Ashwin Karthik VK & Giridhar Ramesh

Lund University

1 Introduction

This project implements a hardware-accelerated convolutional neural network (CNN) for handwritten digit classification on an FPGA. The target application is the MNIST benchmark a standard dataset of 28×28 greyscale images representing handwritten digits from 0 to 9 deployed on the PYNQ-Z1 and RFSoc2x2 development board.

The motivation behind this work is to explore the advantages of moving inference workloads from a general-purpose processor onto dedicated programmable logic. FPGAs offer the ability to exploit massive parallelism and fine-grained datapath control, resulting in lower latency and reduced power consumption compared to software only solutions running on the ARM Cortex-A9 processor embedded in the Zynq device.

The accelerator is written entirely in SystemVerilog RTL and interfaces with the host processor over AXI4-Stream, allowing it to integrate cleanly into the PYNQ software framework. A hardware-folding strategy is employed to fit a 32-filter CNN within the constrained BRAM, DSP, and LUT budget of the xc7z020 device: rather than instantiating all 32 convolutional filters simultaneously, the design time-multiplexes them across 4 sequential fold passes, each processing 8 filters in parallel. This significantly reduces resource usage while retaining correctness.

The report covers the system architecture, RTL implementation decisions, resource utilisation, and classification results obtained from hardware testing.

2 Architecture and Design Choices

The accelerator follows a modular, pipelined architecture built around four cooperating RTL modules: a top-level controller (`CNN_Top_small_PYNQ`), a convolutional filter bank (`Conv_Filter_Bank_0_to_7`), a max-pooling bank (`MaxPool_Bank_0_to_7`), and a fully connected layer (`FC_Layer_Bank_0_to_7`). Together these modules implement the full inference pipeline from raw pixel input to a predicted digit output.

The central design choice is **hardware folding**. Rather than instantiating all 32 convolutional filters in parallel which would exceed the LUTs and DSP budget of the PYNQ Z1, the design processes 8 filters per pass over 4 sequential fold iterations. This reduces logic and memory requirements by a factor of 4 at the cost of latency, a trade-off well suited to the throughput requirements of a one-image-at a time inference system.

All modules share a single clock domain and communicate through registered handshake signals. The host processor streams a 28×28 pixel image into the accelerator over AXI4-Stream, and the accelerator returns the predicted digit (0–9) as a single byte over the same interface once all four fold passes are complete. This keeps the host-facing interface simple and compatible with the PYNQ DMA infrastructure.

2.1 Convolution Architecture

The convolution engine is implemented as a 1-D processing element (PE) array, inspired by the row-stationary dataflow described by Li et al. [**?**]. Rather than computing the full 2-D convolution in a single pass, the design feeds the input image one kernel-row at a time. Three consecutive image rows are streamed into the PE bank, and each PE accumulates partial sums across the three kernel rows before producing a final output pixel. This approach naturally maps to the row sequencer FSM in `CNN_Top`, which controls when each row is presented to the conv bank.

Datapath

Each of the 8 parallel datapaths computes a 3×3 int8 convolution using LUT-based multipliers rather than DSP slices. The accumulator is 19 bits wide, sized to prevent overflow across the nine multiply-accumulate operations without requiring saturation logic mid-accumulation. After all three kernel rows are processed, the raw 19-bit sum passes through a fixed-point requantisation pipeline:

1. A 19-bit bias value is added.
2. The biased sum is multiplied by a per-filter `Q_FIXED` coefficient (32-bit) using a single DSP.
3. The 32-bit result is right-shifted by `RSHIFT` bits and clamped to the int8 range, producing the final `out_pixel[7:0]`.

Design Trade-offs

- **Area:** Using LUT multipliers instead of DSPs for the MAC datapath keeps DSP consumption to 4 (used exclusively for requantisation), at the cost of 669 LUTs and 551 FFs per bank. Precision-matched 19-bit accumulators and 11-bit intermediate clamping avoid wider datapaths that would increase LUT usage further.
- **Performance:** The 1-D PE processes one kernel row per pass at 3 cycles per pixel. This is accepted because the full MNIST inference completes in under 1 ms at the operating clock frequency, which satisfies the throughput requirement for this application.
- **Power:** Narrow datapaths and LUT-based MACs reduce switching activity compared to DSP48-heavy implementations. Combined with zero BRAM usage in the conv datapath itself (accumulators infer as registers), this minimises both dynamic and static power in the filter bank.

2.2 Max-Pooling Architecture

The max-pooling layer applies a 2×2 sliding window with stride 2 to each 26×26 convolution output, reducing it to a 13×13 feature map per channel. Rather than buffering entire rows, the design uses a two-row parity scheme where `pool_active` is asserted only when a valid adjacent row pair is available, keeping logic usage minimal. This stateless approach avoids any additional BRAM allocation and adds negligible latency to the pipeline.

2.3 Fully Connected Layer Architecture

The FC layer is implemented as a bank of 8 parallel MAC engines, one per active filter channel, each computing the dot product of a 169-element pooled feature vector against a 169×10 weight matrix. Outputs accumulate into ten 32-bit class score registers per engine across all four fold passes, after which the fold accumulator in `CNN_Top` reduces the 8 per-channel partial sums into a single set of ten final scores.

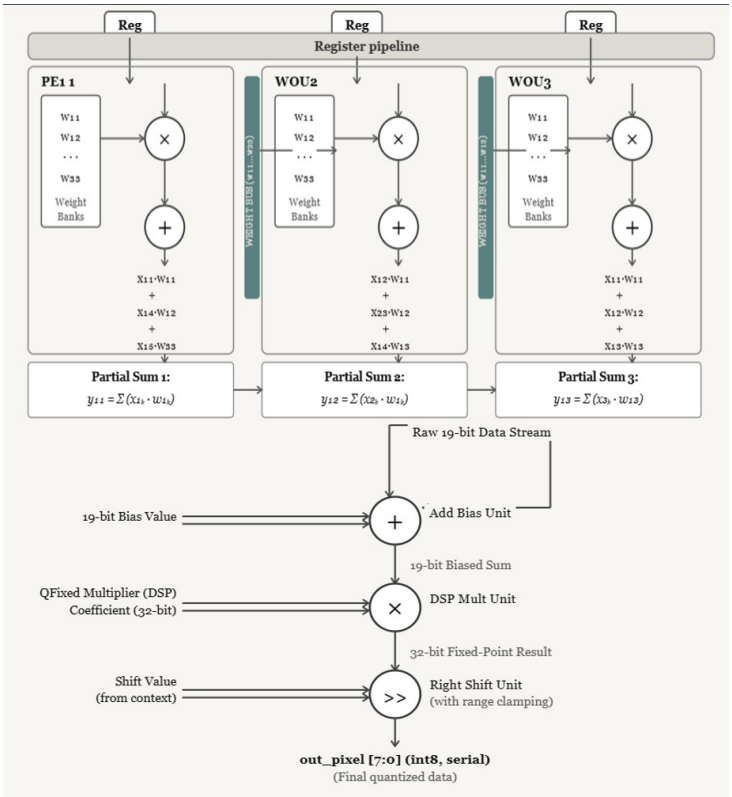


Figure 1: Convolution architecture showing the 1-D PE array, partial sum accumulation, and the requantisation pipeline (bias addition, DSP multiply, right-shift with clamping).

Datapath

Each engine holds two BRAM weight ROMs (`rom_lo` and `rom_hi`), serving two neuron groups simultaneously from a single read address. Two DSP48E1 slices with MREG=1 pipelining multiply a zero-corrected pool pixel against the two weights each cycle, accumulating into a pair of the ten 32-bit output registers. The 2-cycle DSP pipeline delay is compensated by a delayed write-enable (`acc_we_qq`), keeping accumulation continuous with no stall cycles. The centralised FSM iterates over 5 neuron pairs across 13 columns per row, processing the full 13×13 feature map over 13 rows before asserting `valid_out`.

Design Trade-offs

- **Area:** Using 2 DSP48E1 slices per engine (16 total) for MAC and storing all weights in BRAM keeps LUT and FF usage to 528 and 507 respectively. The same 8 engines serve all 32 filters across four fold passes, avoiding the cost of 32 independent datapaths.
- **Performance:** Processing 5 neuron pairs sequentially is accepted because the FC layer operates on a post-pooling 13×13 input, contributing only a small fraction of overall inference latency.
- **Power:** Storing the 169×10 weight matrix in block RAM rather than flip-flops eliminates switching activity across approximately 1,690 FFs, reducing dynamic power in the FC datapath.

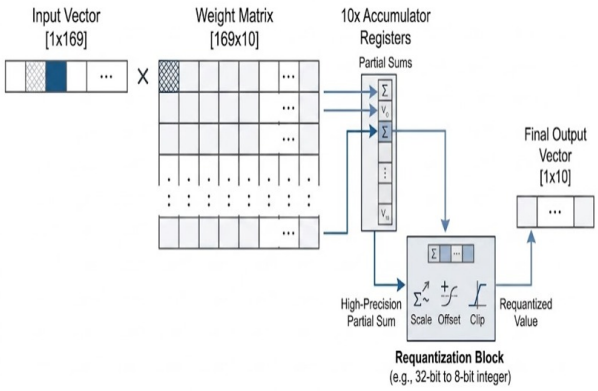


Figure 2: Fully connected layer architecture showing the input vector, weight matrix stored in dual BRAM ports, 10 accumulator registers, and the requantisation block (scale, offset, clip).

3 Parallel Architecture and Hardware Folding

The initial design explored a fully parallel architecture in which all 32 convolutional filters were instantiated simultaneously, each with its own dedicated datapath, weight storage, and accumulator bank. While this maximised throughput, it proved impractical for the target device: the LUT, FF, BRAM, and DSP utilisation far exceeded the available resources on the Zynq xc7z020, making the design unroutable.

To resolve this, the architecture was folded by a factor of 4. Rather than 32 independent datapaths, the design instantiates 8 parallel filter engines and time-multiplexes them across 4 sequential passes (`fold_idx` = 0 to 3). In each pass, the same physical hardware behaves as a different set of 8 logical filters, determined by the `fold_idx` value. This reduces resource usage by approximately 4× while retaining the parallelism benefit of processing 8 filters simultaneously.

The mapping between `fold_idx` and the logical filters assigned to each of the 8 physical slots is shown in Table 1.

`fold_idx`	**Slot 0**	**Slot 1**	**Slot 2**	**Slot 3**	**Slot 4**	**Slot 5**	**Slot 6**	**Slot 7**
`2'd0`	Filter 1	Filter 2	Filter 3	Filter 4	Filter 5	Filter 6	Filter 7	Filter 8
`2'd1`	Filter 9	Filter 10	Filter 11	Filter 12	Filter 13	Filter 14	Filter 15	Filter 16
`2'd2`	Filter 17	Filter 18	Filter 19	Filter 20	Filter 21	Filter 22	Filter 23	Filter 24
`2'd3`	Filter 25	Filter 26	Filter 27	Filter 28	Filter 29	Filter 30	Filter 31	Filter 32

Table 1: Fold index to logical filter mapping across the 8 physical datapath slots.

4 Results and Discussion

4.1 Grade 3 : Single CNN IP on PYNQ-Z1

The first implementation target required deploying a single CNN accelerator IP on the PYNQ-Z1.

Classification Accuracy

Testing the accelerator against 1000 MNIST test images produced a classification accuracy of **96.2%**, demonstrating that the fixed-point quantisation and hardware-folding strategy introduce no meaningful loss relative to the floating-point software model.

Post-Implementation Resource Utilisation

The post-implementation resource utilisation reported by Vivado is shown in Table 2. All resources remain well within the budget of the xc7z020 device, confirming that the folded architecture successfully fits the 32-filter CNN onto the target hardware.

Resource	**Utilisation**	**Available**	**Utilisation %**
LUT	13255	53200	24.92
FF	23475	106400	22.06
BRAM	16.50	140	11.79
DSP	65	220	29.55
IO	75	125	60.00
BUFG	1	32	3.13

Table 2: Post-implementation resource utilisation on Xilinx Zynq xc7z020clg400-1.

The design occupies under 25% of available LUTs and FFs, and under 30% of DSP slices, leaving substantial headroom. The IO utilisation of 60% is expected given the AXI4-Stream and debug ports connected through the PYNQ constraint file.

Performance and Power

The accelerator operates at a clock frequency of **41.6 MHz** post-implementation, meeting timing closure on the xc7z020. Total on-chip power consumption is **0.156 W**, reflecting the efficiency gains from narrow datapaths, LUT-based MACs, and the folded architecture reducing the number of simultaneously active resources.

4.2 Grade 4 : Batch Processing with Router and Debugging on PYNQ-Z1

The Grade 4 target extended the single-image design to support batch inference: multiple images are sent in a single DMA transfer and a hardware router acting as a FIFO demultiplexes the stream, feeding each 784-byte image to the CNN IP in sequence and returning the predicted digits back to the host in order. This eliminates the per-image DMA setup overhead and allows the PS to process a full batch with a single transfer call.

Debugging Methodology

A systematic verification flow was developed to validate the RTL against a behavioural software model. Intermediate feature maps were extracted at each stage of the pipeline and compared output-by-output:

- **Convolution layer:** RTL outputs matched the behavioural model exactly across all 32 filters and all test images.

- **Max-pooling layer:** Results were in perfect agreement with the software reference.
- **Fully connected layer:** A slight numerical variation was observed between the RTL and behavioural model, attributed to fixed-point rounding differences in the DSP-based MAC accumulation. The variation was small enough to not affect the final classification decision.

Classification Accuracy

Batch inference on PYNQ-Z1 achieved the same **96.2%** accuracy over 1000 MNIST images as the single-image Grade 3 implementation, confirming that the router and batch handling introduce no additional classification error.

Post-Implementation Resource Utilisation

Resource	Utilisation	Available	Utilisation %
LUT	21700	53200	40.79
LUTRAM	1397	17400	8.03
FF	32892	106400	30.91
BRAM	3.50	140	2.50
DSP	65	220	29.55
BUFG	1	32	3.13

Table 3: Post-implementation resource utilisation for Grade 4 on Xilinx Zynq xc7z020clg400-1.

The increase in LUT and FF usage compared to Grade 3 reflects the addition of the router and batch control logic. All resources remain within the device budget, with BRAM usage notably lower due to optimisations in the routing infrastructure.

5 Future Work

The current design is area-optimised, which comes at the cost of latency. The convolution and max-pooling layers remain idle while the FC layer runs, and the system waits for a full inference to complete before accepting a new image. A lightweight ready/busy handshake between layers would allow each stage to begin processing the next image as soon as it is free, overlapping computation across images and improving throughput without increasing resource usage.